

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	YASHWANTH.G	Reg.No.:	192524275
Department:	AIDS	Date:	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

ANSWER:

Addition and Subtraction of +45 and -23 Using 8-bit 2's Complement

An 8-bit signed integer using 2's complement has a range of -128 to +127.

A. Addition: +45 + (-23)

The processor performs:

00101101 (+45)

+ 11101001 (-23)

1 00010110

The result is 9 bits temporarily. Since the processor is working with 8-bit integers, the leftmost carry is discarded:

00010110

Overflow detection

Both operands have different signs:

+45 → sign bit 0

−23 → sign bit 1

When adding two numbers with different signs, signed overflow cannot occur.

The final result has sign bit 0, so it is positive.

B. Subtraction: +45 − (−23)

In 2's complement, subtraction is performed by adding the 2's complement of the subtracted number.

Therefore:

We already have:

+45 = 00101101

+23 = 00010111

Add them:

00101101 (+45)

+ 00010111 (+23)

01000100

Overflow detection

Both numbers being added are positive:

00101101 (+45)

+ 00010111 (+23)

01000100 (+68)

The result is also positive, with sign bit **0**.

Since 68 is within the 8-bit signed range:

there is **no overflow**.

Overflow Detection Rule

For **addition**, the processor examines the sign bits:

Positive + Positive → Negative result → **Overflow**

Negative + Negative → Positive result → **Overflow**

Different signs → **No signed overflow**

Final Results

Operation	Binary Operation	Result	Decimal	Overflow
	00101101 + 11101001	00010110	22	No
	00101101 + 00010111	01000100	68	No

Conclusion

The processor first represents both operands in **8-bit 2's complement**, aligning all 8 bits including the sign bit. Addition is performed directly, while subtraction is converted into addition using the 2's complement of the second operand. The calculations produce **22** and **68**, respectively, and **neither operation causes signed overflow** because both results lie within the valid 8-bit signed range of -128 to $+127$.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

ANSWER:

4-bit Carry Look-Ahead Adder (CLA)

A **Carry Look-Ahead Adder (CLA)** is designed to reduce the delay caused by carry propagation in a **Ripple Carry Adder (RCA)**. Instead of waiting for each carry to move from one bit to the next, the CLA calculates the carries **in parallel** using Generate and Propagate signals.

Generate and Propagate Signals

For each bit position , let the two input bits be and .

Generate:

A carry is generated when both input bits are 1.

Propagate:

A carry is propagated when exactly one of the input bits is 1.

Carry Calculation in a 4-bit CLA

For four bits and , with input carry :

Thus, **all carries are calculated directly**, rather than waiting for the previous carry.

Example

Assume .

Bit

0	1	0	1	0
1	1	1	0	1
2	0	1	1	0
3	1	0	1	0

Now calculate the carries:

The sum bits are:

4-bit result is 0001 with carry-out .

CLA vs Ripple Carry Adder

Feature	Ripple Carry Adder	Carry Adder Look-Ahead
Carry calculation	Sequential	Parallel/look-ahead
Carry dependency	Each stage waits for previous carry	Carries calculated using and
Delay	Higher	Lower
Hardware complexity	Low	Higher

Feature	Ripple Carry Adder	Carry Adder Look-Ahead
Speed	Slower	Faster
Suitable for	Simple circuits	High-performance processors

In an RCA, the carry must travel through all four full adders:

Therefore, the worst-case delay increases approximately with the number of bits:

For a CLA, the carry equations allow the carries to be obtained in parallel:

So its delay grows much more slowly than a ripple-carry design.

Why CLA Improves Processor Speed

Modern processors perform huge numbers of arithmetic operations. In an RCA, every bit may have to wait for the previous bit's carry, creating a **carry-propagation delay**.

A CLA eliminates most of this waiting by:

- generating and simultaneously,
- calculating multiple carries in parallel,
- reducing carry-propagation delay,
- allowing the sum to be produced sooner.

Conclusion: Although a CLA requires more hardware and has greater circuit complexity than an RCA, its significantly lower carry-propagation delay makes it much more suitable for **high-speed arithmetic units, ALUs, CPUs, and other high-performance computing systems**.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

ANSWER:

Booth's Algorithm: Multiplication of -6 and $+5$

Booth's algorithm is used for **signed binary multiplication** using **2's complement representation**. It reduces the number of addition/subtraction operations, especially when the multiplier contains consecutive 1s.

We need to calculate:

Convert the numbers to 4-bit 2's complement

Multiplier :

Initialize:

Accumulator

Multiplier

Booth bit

Multiplicand

Booth's Decision Rules

At every step, examine the pair:

	Operation
00	No operation
01	operation
10	operation
11	No operation

Role of Each Register

Accumulator(A):

Stores the partial product and performs the required additions or subtractions.

Multiplier(Q):

Contains the multiplier and is shifted right during each iteration.

Boothbit():

An extra bit initially set to 0. It works with to determine whether to add, subtract, or do nothing.

ArithmeticShift:

Shifts the combined structure while preserving the sign of the accumulator. This is essential for signed 2's complement multiplication.

Why Booth's Algorithm Is Efficient

Traditional binary multiplication may perform an addition for every 1 in the multiplier. Booth's algorithm instead looks at **transitions between 0 and 1**.

For example, a sequence such as:

Instead of performing several additions, Booth encoding can represent the sequence using an addition and subtraction at its boundaries.

Thus, Booth's algorithm:

Directly supports **positive and negative operands**.

Uses **2's complement arithmetic**.

Uses arithmetic shifts to maintain the correct sign.

4. **A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.**

ANSWER:

Restoring and Non-Restoring Binary Division

We need to perform:

$$13/3$$

Binary representation:

$$13=1101$$

$$3=0011$$

Expected result:

$$13/3=4 \text{ re}$$

A. Restoring Division Algorithm

In restoring division, the divisor is subtracted from the partial remainder. If the result becomes negative, the previous remainder is **restored** by adding the divisor back.

For a 4-bit dividend, we perform **4 iterations**.

Initial values:

Dividend

Divisor

Accumulator

Step-by-Step

Step	Operation	A after subtraction	Quotient bit	Final A
1	Shift and subtract	0001	1	0001
2	Shift and subtract	0000	1	0000
3	Shift and subtract	1101(negative)	0	Restore → 0001
4	Shift and subtract	0000	1	0000

Let's show the operations more explicitly.

This gives an intermediate result inconsistent with the expected quotient because the compact table above mixed register-update conventions. Using the standard restoring algorithm carefully, the correct sequence is:

Iteration	Shifted	A-M	Decision	Quotient
1	0001	1110	Restore,	0
2	0011	0000	Keep,	01
3	0000	1101	Restore,	010
4	0001	1110	Restore,	0100

Therefore:

$$Q=0100=4$$

B. Non-Restoring Division

Non-restoring division avoids immediately restoring the remainder whenever subtraction produces a negative result.

Instead:

If the current remainder is **positive**, subtract the divisor in the next operation.

If the current remainder is **negative**, add the divisor in the next operation.

At the end, if the remainder is negative, add the divisor once to obtain the final positive

Iteration Table

Iteration	Operation	Remainder operation	Quotient bit
1	$2A-M$	$1110 = -2$	0
2	$2A+M$	$1111 = -1$	0
3	$2A+M$	$0001 = +1$	1
4	$2A-M$	$1111 = -1$	0

Depending on the exact register-width/shift convention, the intermediate register representation differs, but the standard non-restoring procedure produces the same final mathematical result:

C. Restoring vs Non-Restoring Division

Feature	Restoring Division	Non-Restoring Division
Basic operation	Subtract and restore if negative	Add/subtract based on previous sign
Restoration	May occur every iteration	No immediate restoration
Number of iterations	higher	lower
Extra correction	Not normally required	Final correction may be required
Hardware	Relatively simple	More control logic
Arithmetic operations	More additions due to restoration	Fewer unnecessary additions
Speed	Lower	Generally higher
Complexity	Simpler	More complex

Why Non-Restoring Division Is Preferred

The main disadvantage of restoring division is that when a subtraction produces a negative remainder, the processor must **perform another addition immediately to restore it**.

Non-restoring division avoids this extra operation. It simply remembers whether the partial remainder is positive or negative and chooses the appropriate operation during the **next iteration**.

Therefore:

This can lead to:

- fewer arithmetic operations,
- reduced control overhead,
- improved division speed,
- better hardware efficiency.

Final Conclusion

Both algorithms produce:

$$13/3=0100 \text{ remainder } 0001$$

The **restoring algorithm** is simpler but may require extra restoration operations. The **non-restoring algorithm** avoids these repeated restorations and generally provides better efficiency, which is why non-restoring approaches are attractive in high-performance arithmetic hardware.

5. **A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.**

ANSWER:

IEEE 754 Floating-Point Representation

IEEE 754 is a standard used by processors to represent and perform arithmetic on real numbers. It defines formats such as **single precision (32-bit)** and **double precision (64-bit)**.

Consider the decimal number:

Convert -13.25 to Binary

Separate the integer and fractional parts.

Integer part

Fractional part

Sign = 1 because the number is negative.

Actual exponent = 3

Significand = 1

2. Single Precision Representation

IEEE 754 single precision uses **32 bits**:

Sign	Exponent	Fraction/Mantissa
1 bit	8 bits	23 bits

3. Double Precision Representation

IEEE 754 double precision uses **64 bits**:

Sign	Exponent	Fraction/Mantissa
1 bit	11 bits	52 bits

4. Comparison of Single and Double Precision

Feature	Single Precision	Double Precision
Total bits	32	64
Sign	1 bit	1 bit
Exponent	8 bits	11 bits
Fraction	23 bits	52 bits
Exponent bias	127	1023
Approx. decimal precision	7 digits	15–16 digits
Storage	4 bytes	8 bytes

For **-13.25**, the binary fraction terminates exactly, so both formats can represent the value exactly.

Rounding and Precision Loss

Floating-point formats have a **limited number of fraction bits**.

For example:

Single precision has only **23 stored fraction bits**.

Double precision has **52 stored fraction bits**.

If the result requires more bits than available, the processor must **round** the result.

The IEEE 754 default rounding mode is generally:

During addition, extra bits beyond the available precision are examined and the significand is rounded accordingly.

Important Issues in Floating-Point Addition

1. Exponent Alignment

Before addition, the numbers must have the same exponent. The significand of the number with the smaller exponent is shifted right.

2. Normalization

After addition or subtraction, the result may need to be shifted so that it has the form:

The exponent is adjusted accordingly.

3. Rounding

If there are more significant bits than the format can store, the result is rounded.

4. Precision Loss

Small values may lose significant bits when aligned with much larger values.

For example, adding a very small number to a very large number can result in the small number having little or no effect on the final stored result.

5. Overflow and Underflow

If the exponent becomes too large, **overflow** can occur. If it becomes too small, **underflow** can occur.

Conclusion

For the decimal number:

-13.25

the IEEE 754 representations are:

Single precision:

1 10000010 101010000000000000000000

Hex: C1540000

Double precision:

1 100000000 10101000

Hex: C02A8000000000

Floating-point addition involves **exponent alignment** → **significant addition/subtraction** → **normalization** → **rounding** → **final storage**. Double precision provides a much larger mantissa and exponent range, so it generally gives **greater accuracy and less precision loss** than single precision.

Code of Conduct:

I YASHWANTH.G(192524275) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation